recording deployments and releases, and using the can-i-deploy tool, to name a few. In this section, we show how to integrate these features.

10.2.1 Configuring consumer and provider versions

If you remember from chapter 4, we were updating the consumer contract version, especially if there were changes to the contract. For a more automated approach, the versions need to be configured in a way that's unique on the release branch or merge to the main branch. One way to do this is using the GitHub commit ID, which is called a Secure Hash Algorithm (SHA) or a hash. Using the GitHub SHA ensures that the contract changes can be easily traced to the relevant code changes.

To follow along in the following hands-on guide, make sure you have a copy of the example GitHub project (https://github.com/mdcruz/pact-js-example), and switch to a new branch called setup-advanced-pact-features. Once you're in the setup-advanced-pact-features branch, the first thing you need to update is the contract-test-sample.yaml file and introduce a new environment variable called `GITHUB_SHA`, which should contain the value of the GitHub SHA. To do this in GitHub Actions, add the following code snippet:

```
env:
  PACT_BROKER_BASE_URL: ${{ secrets.PACT_BROKER_BASE_URL }}
  PACT_BROKER_TOKEN: ${{ secrets.PACT_BROKER_TOKEN }}
  GITHUB_SHA: ${{ github.sha }}
```

Next, let's update command `publish:pact` in the package.json file to use the `GITHUB_SHA` environment variable instead of the manual version:

```
"scripts": {
     "publish:pact": "pact-broker publish ./pacts
--consumer-app-version=$GITHUB_SHA --tag=main
--broker-base-url=$PACT_BROKER_BASE_URL
--broker-token=$PACT_BROKER_TOKEN"
}
```

This ensures that when the contract is published, the version will point to the commit ID that introduces the change. To test the change, feel free to commit the changes to your local GitHub branch, push the changes remotely to your own fork, and create a pull request.